Sei un ingegnere senior esperto di Python e NVIDIA CUDA-Q.

I file dal `calcolo_07_codice_e_risultato.py` al `calcolo_21_codice_e_risultato.py` contengono ancora gravi errori: `cudaq.mz()` dentro i kernel, formule sbagliate (es. 9.2% invece di 95.9%, 20.2 kW invece di 24.5 kW), e JSON hardcoded.

Il tuo compito è sostituire il contenuto di TUTTI questi file (dal 7 al 21) con il codice corretto che ti fornisco qui sotto.

NON toccare i file dal 1 al 6 (sono già corretti).

Ecco il codice corretto per i calcoli dal 7 al 21:

--- CALCOLO #07 ---
```python
import cudaq
import numpy as np
from typing import Dict, Any

@cudaq.kernel
def hopfield_safety_kernel(theta_stress: float, theta_press: float):
    q = cudaq.qvector(4)
    for i in range(4):
        cudaq.h(q[i])
    cudaq.ry(theta_stress, q[0])
    cudaq.ry(theta_press, q[1])
    cudaq.x.ctrl(q[0], q[2])
    cudaq.x.ctrl(q[1], q[3])

def esegui_calcolo_07_hopfield(corsie_attive: int = 6, pressione_pattini_bar: float = 185.0, stress_strutturale_mpa: float = 142.5) -> Dict[str, Any]:
    c_att = max(1, int(corsie_attive))
    press = float(np.clip(pressione_pattini_bar, 50.0, 300.0))
    stress = max(1.0, float(np.clip(stress_strutturale_mpa, 20.0, 300.0)))
    theta_s = float(np.clip((stress / 250.0) * np.pi, 0.1, np.pi - 0.1))
    theta_p = float(np.clip((press / 250.0) * np.pi, 0.1, np.pi - 0.1))
    counts = cudaq.sample(hopfield_safety_kernel, theta_s, theta_p, shots=100)
    stato = counts.most_probable()
    limite_acciaio = 235.0
    fattore_sicurezza = round(float(limite_acciaio / stress), 2)
    return {
        "calcolo": "Quantum Hopfield Networks - Safety Scaffalature Intensive",
        "corsie_attive": c_att,
        "stress_strutturale_mpa": stress,
        "fattore_sicurezza": fattore_sicurezza,
        "conformita_asrs": "CONFORME NORMA EN 15635" if fattore_sicurezza >= 1.5 else "ALLARME SOVRACCARICO",
```

```
    }
```

--- CALCOLO #08 ---

```
python
import cudaq
import numpy as np
from typing import Dict, Any

@cudaq.kernel
def qwalk_tsp_kernel(theta_dist: float):
    q = cudaq.qvector(4)
    for i in range(4):
        cudaq.h(q[i])
        cudaq.ry(theta_dist * (i + 1), q[i])
    cudaq.x.ctrl(q[0], q[1])
    cudaq.x.ctrl(q[1], q[2])
    cudaq.x.ctrl(q[2], q[3])

def esegui_calcolo_08_tsp(nodi_prelievo_count: int = 4, distanza_totale_metri:
float = 340.0) -> Dict[str, Any]:
    dist = max(10.0, float(distanza_totale_metri))
    theta_dist = float(np.clip((dist / 1000.0) * np.pi, 0.1, np.pi - 0.1))
    counts = cudaq.sample(qwalk_tsp_kernel, theta_dist, shots=100)
    stato_rotta = counts.most_probable()
    nodi_base = ["PLT_A (Corsia 1)", "PLT_B (Corsia 3)", "PLT_C (Corsia 2)",
"PLT_D (Baia Scarico)"]
    shift_idx = int(stato_rotta, 2) % len(nodi_base)
    sequenza_ottimizzata = nodi_base[shift_idx:] + nodi_base[:shift_idx]
    risparmio_metri = round(float(dist * (0.15 + (int(stato_rotta, 2) / 64.0))),
1)
    return {
        "calcolo": "Quantum Walk Route Exploration (TSP) - Picking & Batching",
        "distanza_nominale_metri": dist,
        "distanza_ottimizzata_metri": round(dist - risparmio_metri, 1),
        "metri_lineari_risparmiati": risparmio_metri,
        "stato_qubit_percorso": stato_rotta,
        "sequenza_tappe_ottimale": sequenza_ottimizzata
```

```
    }
```

--- CALCOLO #09 ---

```
python
```

```
import cudaq
import numpy as np
from typing import Dict, Any

@cudaq.kernel
def qgnn_batching_kernel(w0: float, w1: float, w2: float, w3: float, w4: float,
w5: float, w6: float, w7: float, densita_grafo: float):
    q = cudaq.qvector(4)
    cudaq.ry(densita_grafo * 0.45, q[0])
    cudaq.ry(densita_grafo * 0.90, q[1])
    cudaq.ry(densita_grafo * 1.35, q[2])
    cudaq.ry(densita_grafo * 1.80, q[3])
    cudaq.rx(w0, q[0])
    cudaq.rz(w4, q[0])
    cudaq.rx(w1, q[1])
    cudaq.rz(w5, q[1])
    cudaq.rx(w2, q[2])
    cudaq.rz(w6, q[2])
    cudaq.rx(w3, q[3])
    cudaq.rz(w7, q[3])
    cudaq.x.ctrl(q[0], q[1])
    cudaq.x.ctrl(q[1], q[2])
    cudaq.x.ctrl(q[2], q[3])
    cudaq.x.ctrl(q[3], q[0])

def esegui_calcolo_09_qgnn(ordini_in_coda: int = 24, densita_affiliazioni_grafo:
float = 0.65) -> Dict[str, Any]:
    n_ord = max(1, int(ordini_in_coda))
    dens = float(np.clip(densita_affiliazioni_grafo, 0.05, 1.0))
    w = [0.45, 0.62, 0.38, 0.71, 0.55, 0.40, 0.68, 0.33]
    counts = cudaq.sample(qgnn_batching_kernel, w[0], w[1], w[2], w[3], w[4],
w[5], w[6], w[7], dens, shots=100)
    stato_cluster = counts.most_probable()
    n_cluster = max(2, min(8, int(stato_cluster, 2) % 6 + 2))
    efficienza = round(min(98.5, max(70.0, 82.0 + dens * 14.5)), 1)
    tempo_sec = round(float((n_ord / n_cluster) * 4.2), 1)
    return {
        "calcolo": "Quantum Graph Neural Network (QGNN) - Order Batching",
        "ordini_in_coda": n_ord,
        "cluster_ottimale_bitstring": stato_cluster,
        "numero_cluster_generati": n_cluster,
        "efficienza_batching_pct": efficienza,
        "tempo_consolidamento_sec": tempo_sec,
        "distribuzione_campionata": dict(counts)
    }
```

--- CALCOLO #10 ---

```python
import cudaq
import numpy as np
from typing import Dict, Any

@cudaq.kernel
def qaoa_3d_loader_kernel(theta_v: float, theta_w: float):
    q = cudaq.qvector(4)
    for i in range(4):
        cudaq.h(q[i])
    cudaq.ry(theta_v, q[0])
    cudaq.ry(theta_w, q[1])
    cudaq.x.ctrl(q[0], q[2])
    cudaq.x.ctrl(q[1], q[3])

def esegui_calcolo_10_loader(colli_da_caricare: int = 33, peso_totale_kg: float
= 24500.0, volume_cassone_m3: float = 86.0) -> Dict[str, Any]:
    c_colli = max(1, int(colli_da_caricare))
    p_tot = max(100.0, float(peso_totale_kg))
    v_cass = max(1.0, float(volume_cassone_m3))
    saturazione_pct = round(min(99.4, (c_colli * 2.5 / v_cass) * 100.0), 1)
    theta_v = float(np.clip((saturazione_pct / 100.0) * np.pi, 0.1, np.pi -
0.1))
    theta_w = float(np.clip((p_tot / 30000.0) * np.pi, 0.1, np.pi - 0.1))
    counts = cudaq.sample(qaoa_3d_loader_kernel, theta_v, theta_w, shots=100)
    stato = counts.most_probable()
    peso_ant = round(float(p_tot * 0.42), 1)
    peso_post = round(float(p_tot * 0.58), 1)
    return {
        "calcolo": "Quantum Approximate Optimization (QAOA) - 3D Volumetric
Loader",
        "colli_allocati": c_colli,
        "volume_cassone_m3": v_cass,
        "saturazione_volumetrica_pct": saturazione_pct,
        "peso_totale_kg": p_tot,
        "ripartizione_assi_kg": {"asse_anteriore": peso_ant, "asse_posteriore":
peso_post},
        "configurazione_qubit_stivaggio": stato,
        "esito_sagoma": "CONFORME CODICE DELLA STRADA (44 t)"
    }
```

--- CALCOLO #11 ---

```python
import cudaq
```

```
import secrets
import hashlib
import numpy as np
from typing import Dict, Any

@cudaq.kernel
def blind_signature_kernel(theta_phase: float):
    q = cudaq.qvector(4)
    for i in range(4):
        cudaq.h(q[i])
        cudaq.rz(theta_phase * (i + 1), q[i])

def esegui_calcolo_11_ecmr(id_spedizione: str = "ECMR-NOV-2026-9921",
destinazione: str = "HUB-INTERPORTO-BOLOGNA", vettore_logistico: str =
"FERCAM-LOGISTICS-IT") -> Dict[str, Any]:
    theta_seed = float((len(id_spedizione) % 8) / 8.0 * np.pi)
    counts = cudaq.sample(blind_signature_kernel, theta_seed, shots=100)
    qpu_entropy = counts.most_probable()
    blinding_factor = secrets.token_hex(16)
    raw_carrier =
f"{vettore_logistico.strip()}::{destinazione.strip()}::BARILLA_SALT_2026"
    carrier_token =
f"OP-CARRIER-{hashlib.sha3_256(raw_carrier.encode('utf-8')).hexdigest()[:16].upp
er()}"
    blinded_payload =
f"{id_spedizione}|{carrier_token}|{qpu_entropy}::BLINDED_BY::{blinding_factor}"
    blind_hash = hashlib.sha3_256(blinded_payload.encode('utf-8')).hexdigest()
    unblind_token =
hashlib.sha3_256(f"{blind_hash}::{blinding_factor}".encode('utf-8')).hexdigest()
    return {
        "calcolo": "Quantum-Enhanced Blind Signatures e-CMR",
        "metodo": "Euristica di Accecamento Crittografico con Seed Quantistico
(Non-QPU Lattice)",
        "id_spedizione": id_spedizione,
        "vettore_pseudonimizzato_gdpr": carrier_token,
        "firma_cieca_hash": blind_hash,
        "token_unblind_notarile": unblind_token,
        "stato_qubit_entropy": qpu_entropy
    }
```

--- CALCOLO #12 ---

```
python
import cudaq
import numpy as np
from typing import Dict, Any
```

```
@cudaq.kernel
def ewl_game_kernel(gamma_j: float, theta_driver: float, theta_plant: float):
    q = cudaq.qvector(4)
    for i in range(2):
        cudaq.h(q[i])
    cudaq.x.ctrl(q[0], q[1])
    cudaq.rz.ctrl(gamma_j, q[0], q[1])
    cudaq.x.ctrl(q[0], q[1])
    cudaq.ry(theta_driver, q[0])
    cudaq.ry(theta_plant, q[1])
    cudaq.x.ctrl(q[0], q[1])
    cudaq.rz.ctrl(-gamma_j, q[0], q[1])
    cudaq.x.ctrl(q[0], q[1])

def esegui_calcolo_12_game_theory(camion_in_coda: int = 14, baie_disponibili:
int = 2, costo_orario_attesa_eur: float = 65.0) -> Dict[str, Any]:
    c_coda = max(1, int(camion_in_coda))
    b_disp = max(1, int(baie_disponibili))
    costo_h = max(10.0, float(costo_orario_attesa_eur))
    theta_d = float(np.clip((c_coda / 20.0) * np.pi, 0.1, np.pi - 0.1))
    theta_p = float(np.clip((b_disp / 6.0) * np.pi, 0.1, np.pi - 0.1))
    counts = cudaq.sample(ewl_game_kernel, 0.785, theta_d, theta_p, shots=100)
    stato = counts.most_probable()
    risparmio_eur = round(float((c_coda * costo_h * 0.45) * (b_disp / 4.0)), 2)
    return {
        "calcolo": "Quantum Game Theory (EWL) - Dock & Slot Scheduler",
        "camion_in_coda": c_coda,
        "baie_disponibili": b_disp,
        "risparmio_costi_sosta_eur": risparmio_eur,
        "stato_equilibrio_nash_qubit": stato,
        "esito_cooperativo": "PARETO-OTTIMALE" if stato in ["0011", "0000"] else
"SUB-OTTIMALE"
    }
```

--- CALCOLO #13 ---

```
python
import cudaq
import numpy as np
from typing import Dict, Any

@cudaq.kernel
def qk_means_traffic_kernel(theta_c: float, theta_b: float):
    q = cudaq.qvector(4)
    cudaq.ry(theta_c, q[0])
```

```
    cudaq.ry(theta_b, q[1])
    cudaq.x.ctrl(q[0], q[2])
    cudaq.x.ctrl(q[1], q[3])

def esegui_calcolo_13_yard(camion_attesa: int = 14, saturazione_buffer: float =
0.75) -> Dict[str, Any]:
    c = max(0, int(camion_attesa))
    sat = float(np.clip(saturazione_buffer, 0.0, 1.0))
    theta_c = float(np.clip((c / 20.0) * np.pi, 0.1, np.pi - 0.1))
    theta_b = float(np.clip(sat * np.pi, 0.1, np.pi - 0.1))
    counts = cudaq.sample(qk_means_traffic_kernel, theta_c, theta_b, shots=100)
    cluster_dominante = counts.most_probable()
    tempo_medio_attesa_min = round(float(c * 3.8 * sat), 1)
    return {
        "calcolo": "Quantum K-Means Clustering - Traffico Piazzale e Buffer",
        "camion_attesa": c,
        "saturazione_buffer": sat,
        "cluster_qubit_traffico": cluster_dominante,
        "tempo_medio_attesa_min": tempo_medio_attesa_min,
        "stato_congestione": "ATTENZIONE - Regolare Flussi Banchina" if sat >
0.70 else "FLUIDO"
    }
```

--- CALCOLO #14 ---

```
python
import cudaq
import numpy as np
from typing import Dict, Any

@cudaq.kernel
def qft_bema_kernel(theta_freq: float):
    q = cudaq.qvector(4)
    for i in range(4):
        cudaq.ry(theta_freq * (i + 1), q[i])
    cudaq.h(q[0])
    cudaq.rz.ctrl(np.pi / 2.0, q[1], q[0])
    cudaq.rz.ctrl(np.pi / 4.0, q[2], q[0])
    cudaq.rz.ctrl(np.pi / 8.0, q[3], q[0])
    cudaq.h(q[1])
    cudaq.rz.ctrl(np.pi / 2.0, q[2], q[1])
    cudaq.rz.ctrl(np.pi / 4.0, q[3], q[1])
    cudaq.h(q[2])
    cudaq.rz.ctrl(np.pi / 2.0, q[3], q[2])
    cudaq.h(q[3])
    cudaq.x.ctrl(q[0], q[3])
```

```
    cudaq.x.ctrl(q[3], q[0])
    cudaq.x.ctrl(q[0], q[3])
    cudaq.x.ctrl(q[1], q[2])
    cudaq.x.ctrl(q[2], q[1])
    cudaq.x.ctrl(q[1], q[2])

def esegui_calcolo_14_qft_bema(rpm_tavola_bema: float = 48.5,
vibrazione_cuscinetti_g: float = 0.36, pre_stiro_pct: float = 250.0) ->
Dict[str, Any]:
    rpm = max(1.0, float(rpm_tavola_bema))
    vib_g = max(0.01, float(vibrazione_cuscinetti_g))
    prestiro = float(np.clip(pre_stiro_pct, 100.0, 350.0))
    f0_hz = round(float(rpm / 60.0), 3)
    theta_freq = float(np.clip(f0_hz * 1.25, 0.1, np.pi - 0.1))
    counts = cudaq.sample(qft_bema_kernel, theta_freq, shots=100)
    bin_dominante = counts.most_probable()
    bin_index = int(bin_dominante, 2)
    freq_armonica_hz = round(float(bin_index * (f0_hz * 2.0)), 2)
    return {
        "calcolo": "Quantum Fourier Transform (QFT) - Bema Wrapping Controller",
        "rpm_tavola": rpm,
        "frequenza_fondamentale_hz": f0_hz,
        "pre_stiro_pct": prestiro,
        "bin_spettrale_rilevato": bin_dominante,
        "frequenza_armonica_stimata_hz": freq_armonica_hz,
        "vibrazione_cuscinetti_g": vib_g,
        "stato_meccanico": "ALLARME MANUTENZIONE PREVENTIVA" if vib_g > 0.45
else "REGOLARE"
```

```
    }
```

--- CALCOLO #15 ---

```
python
import cudaq
import numpy as np
from typing import Dict, Any

@cudaq.kernel
def qsvm_film_kernel(theta_t: float, theta_p: float):
    q = cudaq.qvector(4)
    cudaq.ry(theta_t, q[0])
    cudaq.ry(theta_p, q[1])
    cudaq.x.ctrl(q[0], q[2])
    cudaq.x.ctrl(q[1], q[3])
```

```
def compute_stress_mpa(tension_n: float = 148.5, thickness_um: float = 23.0,
width_mm: float = 1000.0) -> float:
    area_mm2 = width_mm * (thickness_um / 1000.0)
    return round(float(tension_n / area_mm2), 2)

def esegui_calcolo_15_qsvm_film(tensione_newton: float = 148.5, pre_stiro_pct:
float = 250.0, spessore_film_micron: float = 23.0) -> Dict[str, Any]:
    t_n = max(1.0, float(tensione_newton))
    prestiro = float(np.clip(pre_stiro_pct, 100.0, 350.0))
    spess_um = max(1.0, float(spessore_film_micron))
    sforzo_mpa = compute_stress_mpa(t_n, spess_um, width_mm=1000.0)
    theta_t = float(np.clip((t_n / 300.0) * np.pi, 0.1, np.pi - 0.1))
    theta_p = float(np.clip((prestiro / 350.0) * np.pi, 0.1, np.pi - 0.1))
    counts = cudaq.sample(qsvm_film_kernel, theta_t, theta_p, shots=100)
    stato = counts.most_probable()
    return {
        "calcolo": "Quantum Support Vector Machines (QSVM) - Tensione Film
Bema",
        "tensione_newton": t_n,
        "pre_stiro_pct": prestiro,
        "spessore_film_micron": spess_um,
        "sforzo_unitario_mpa": sforzo_mpa,
        "stato_fasciatura": "ALLARME ROTTURA FILM" if sforzo_mpa > 18.0 else
"REGOLARE"
    }
```

-- CALCOLO #16 --

```
python
import cudaq
import numpy as np
from typing import Dict, Any

@cudaq.kernel
def lgv_routing_kernel(theta_navette: float, theta_nodi: float):
    q = cudaq.qvector(4)
    for i in range(4):
        cudaq.h(q[i])
    cudaq.ry(theta_navette, q[0])
    cudaq.ry(theta_nodi, q[1])
    cudaq.x.ctrl(q[0], q[2])
    cudaq.x.ctrl(q[1], q[3])

def esegui_calcolo_16_lgv_fleet(navette_in_missione: int = 18,
nodi_critici_congestione: int = 3) -> Dict[str, Any]:
    nav = max(1, int(navette_in_missione))
```

```
    nodi = max(0, int(nodi_critici_congestione))
    theta_nav = float(np.clip((nav / 30.0) * np.pi, 0.1, np.pi - 0.1))
    theta_nod = float(np.clip((nodi / 10.0) * np.pi, 0.1, np.pi - 0.1))
    counts = cudaq.sample(lgv_routing_kernel, theta_nav, theta_nod, shots=100)
    stato = counts.most_probable()
    fluidita_pct = round(float(max(30.0, 98.5 - (nodi * 14.5) - (nav * 1.5))),
1)
    tempo_transito_sec = round(140.0 * (100.0 / fluidita_pct), 1)
    return {
        "calcolo": "Dynamic Graph QAOA & QRL - Routing & Traffic Engine LGV",
        "navette_attive": nav,
        "nodi_critici": nodi,
        "indice_fluidita_flotta_pct": fluidita_pct,
        "tempo_medio_transito_sec": tempo_transito_sec,
        "stato_qubit_traffico": stato
    }
```

--- CALCOLO #17 ---

```
python
import cudaq
import numpy as np
from typing import Dict, Any

@cudaq.kernel
def bipartite_matching_kernel(theta_soc: float, theta_temp: float):
    q = cudaq.qvector(4)
    cudaq.ry(theta_soc, q[0])
    cudaq.ry(theta_temp, q[1])
    cudaq.x.ctrl(q[0], q[2])
    cudaq.x.ctrl(q[1], q[3])

def esegui_calcolo_17_bms_lgv(missioni_urgenti: int = 8, soc_medio_flotta_pct:
float = 64.5, temperatura_batterie_c: float = 34.2) -> Dict[str, Any]:
    miss = max(1, int(missioni_urgenti))
    soc = float(np.clip(soc_medio_flotta_pct, 10.0, 100.0))
    temp = float(np.clip(temperatura_batterie_c, 15.0, 60.0))
    theta_s = float((soc / 100.0) * np.pi)
    theta_t = float((temp / 50.0) * np.pi)
    counts = cudaq.sample(bipartite_matching_kernel, theta_s, theta_t,
shots=100)
    stato = counts.most_probable()
    efficienza_matching = round(float(min(99.0, max(50.0, (soc * 0.85) + ((45.0
- temp) * 0.5)))), 1)
    return {
        "calcolo": "Quantum Bipartite Matching - Task Allocation Engine",
```

```
        "missioni_urgenti": miss,
        "soc_medio_flotta_pct": soc,
        "temperatura_batterie_c": temp,
        "efficienza_accoppiamento_pct": efficienza_matching,
        "stato_qubit_allocazione": stato
    }
```

--- CALCOLO #18 ---

```
python
import cudaq
from cudaq import spin
import numpy as np
from scipy.optimize import minimize
from typing import Dict, Any

@cudaq.kernel
def vqe_robot_ansatz(theta_j1: float, theta_j2: float, theta_j3: float):
    q = cudaq.qvector(4)
    cudaq.x(q[0])
    cudaq.ry(theta_j1, q[1])
    cudaq.ry(theta_j2, q[2])
    cudaq.ry(theta_j3, q[3])
    cudaq.x.ctrl(q[1], q[0])
    cudaq.x.ctrl(q[2], q[1])
    cudaq.x.ctrl(q[3], q[2])

def esegui_calcolo_18_vqe_robot(coppia_motori_nm: float = 185.0,
pressione_vuoto_bar: float = -0.78, corrente_joint_ampere: float = 14.2) ->
Dict[str, Any]:
    c_nm = max(10.0, float(coppia_motori_nm))
    p_bar = float(np.clip(pressione_vuoto_bar, -1.0, 0.0))
    i_amp = max(1.0, float(corrente_joint_ampere))
    scale_coppia = float(c_nm / 200.0)
    H = -1.05 * spin.z(0) + (0.45 * scale_coppia) * spin.z(1) + 0.35 * spin.z(2)
+ 0.25 * spin.z(3)
    H += 0.20 * (spin.x(0) * spin.x(1) + spin.y(0) * spin.y(1))
    def cost_func(params):
        return cudaq.observe(vqe_robot_ansatz, H, float(params[0]),
float(params[1]), float(params[2])).expectation()
    opt_res = minimize(cost_func, [0.35, 0.55, 0.42], method='COBYLA',
options={'maxiter': 20})
    energia = round(float(opt_res.fun), 4)
    vuoto_conforme = p_bar <= -0.65
    vibrazione_giunti_mm = round(float(0.12 + abs(energia) * 0.04), 3)
    return {
```

```
        "calcolo": "Variational Quantum Eigensolver (VQE) - Robot Joint
Kinematics",
        "metodo": "VQE Variazionale Reale con Hamiltoniana di Spin (4 Qubit)",
        "coppia_motori_nm": c_nm,
        "pressione_vuoto_bar": p_bar,
        "corrente_joint_ampere": i_amp,
        "energia_ground_state_trovata": energia,
        "angoli_ottimi_rad": [round(float(opt_res.x[0]), 3),
round(float(opt_res.x[1]), 3), round(float(opt_res.x[2]), 3)],
        "tenuta_vuoto_pinza": "TENUTA OTTIMALE" if vuoto_conforme else "ALLARME
VUOTO INSUFFICIENTE",
        "vibrazione_giunti_mm": vibrazione_giunti_mm,
        "stato_manutenzione": "REGOLARE" if abs(energia) < 2.5 else "ISPEZIONE
GIUNTI RICHIESTA"
    }
```

--- CALCOLO #19 ---

```python
import cudaq
import numpy as np
from typing import Dict, Any

@cudaq.kernel
def knapsack_charge_kernel(theta_p: float, theta_t: float):
    q = cudaq.qvector(4)
    cudaq.ry(theta_p, q[0])
    cudaq.ry(theta_t, q[1])
    cudaq.x.ctrl(q[0], q[2])
    cudaq.x.ctrl(q[1], q[3])

def esegui_calcolo_19_fast_charge(potenza_erogata_totale_kw: float = 87.5,
temp_piastre_c: float = 42.5, livello_supercondensatori_pct: float = 68.0) ->
Dict[str, Any]:
    p_kw = max(0.0, float(potenza_erogata_totale_kw))
    t_c = float(temp_piastre_c)
    sc_pct = float(np.clip(livello_supercondensatori_pct, 0.0, 100.0))
    theta_p = float(np.clip((p_kw / 120.0) * np.pi, 0.1, np.pi - 0.1))
    theta_t = float(np.clip((t_c / 80.0) * np.pi, 0.1, np.pi - 0.1))
    counts = cudaq.sample(knapsack_charge_kernel, theta_p, theta_t, shots=100)
    stato = counts.most_probable()
    picco_tagliato = round(float(p_kw * 0.28), 1)
    energia_recuperata = round(float(p_kw * (sc_pct / 100.0) * 0.45), 2)
    return {
        "calcolo": "Quantum Knapsack & Microgrid Peak Shaving",
        "potenza_erogata_totale_kw": p_kw,
```

```
        "temp_piastre_c": t_c,
        "picco_tagliato_kw": picco_tagliato,
        "energia_recuperata_kwh": energia_recuperata,
        "modello_accumulo": "Curva lineare approssimata (validita SoC nominale
20%-80%)",
        "avviso_tecnico_supercap": "Per SoC <20% o >80% applicare curva
esponenziale non lineare Maxwell/Skeleton",
        "stato_termico": "SICURO - Carica Ultra-Fast Abilitata" if t_c <= 65.0
else "ALLARME SOVRATEMPERATURA",
        "configurazione_qubit_allocata": stato
    }
```

--- CALCOLO #20 ---

```python
import cudaq
import numpy as np
from typing import Dict, Any

@cudaq.kernel
def woodpecker_svc_kernel(theta_u: float, theta_f: float):
    q = cudaq.qvector(4)
    cudaq.ry(theta_u, q[0])
    cudaq.ry(theta_f, q[1])
    cudaq.x.ctrl(q[0], q[2])
    cudaq.x.ctrl(q[1], q[3])

def esegui_calcolo_20_woodpecker(throughput_pallet_ora: int = 280,
umidita_legno_pct: float = 13.4, forza_deformazione_pattini_n: float = 3400.0)
-> Dict[str, Any]:
    th_plt = max(1, int(throughput_pallet_ora))
    umid = float(np.clip(umidita_legno_pct, 5.0, 40.0))
    forza = max(100.0, float(forza_deformazione_pattini_n))
    theta_u = float(np.clip((umid / 30.0) * np.pi, 0.1, np.pi - 0.1))
    theta_f = float(np.clip((forza / 5000.0) * np.pi, 0.1, np.pi - 0.1))
    counts = cudaq.sample(woodpecker_svc_kernel, theta_u, theta_f, shots=100)
    stato = counts.most_probable()
    delta_umidita = max(0.0, 22.0 - umid)
    tasso_scarto_stimato_pct = round(float(max(0.5, delta_umidita * 0.18)), 2)
    pallet_scartati_stima = int(round(th_plt * (tasso_scarto_stimato_pct /
100.0)))
    esito_conforme = (umid <= 18.0) and (forza > 2800.0)
    return {
        "calcolo": "Quantum Support Vector Classifier - Woodpecker Pallet
Integrity",
        "throughput_pallet_ora": th_plt,
```

```
        "umidita_legno_pct": umid,
        "forza_deformazione_pattini_n": forza,
        "tasso_scarto_stimato_pct": tasso_scarto_stimato_pct,
        "pallet_scartati_ora_stima": pallet_scartati_stima,
        "esito_collaudo": "APPROVATO - Pallet Conforme Standard E80" if
esito_conforme else "SCARTATO - Fuori Tolleranza",
        "stato_qubit_decisione": stato,
        "distribuzione_campionata": dict(counts)
    }
```

--- CALCOLO #21 ---

```
python
import cudaq
import numpy as np
from typing import Dict, Any

@cudaq.kernel
def qem_raptor_kernel(theta_c: float):
    q = cudaq.qvector(4)
    for i in range(4):
        cudaq.h(q[i])
        cudaq.ry(theta_c * (i + 1), q[i])
    cudaq.x.ctrl(q[0], q[1])
    cudaq.x.ctrl(q[2], q[3])

def esegui_calcolo_21_raptor(grado_qualita_stampa_iso: str = "GRADO_A",
contrasto_ottico_pct: float = 84.5) -> Dict[str, Any]:
    contrasto = float(np.clip(contrasto_ottico_pct, 10.0, 100.0))
    theta_c = float((contrasto / 100.0) * np.pi)
    counts = cudaq.sample(qem_raptor_kernel, theta_c, shots=100)
    stato = counts.most_probable()
    bonus_grado = 10.0 if grado_qualita_stampa_iso == "GRADO_A" else (5.0 if
grado_qualita_stampa_iso == "GRADO_B" else 0.0)
    affidabilita = round(min(99.9, (contrasto * 1.05) + bonus_grado), 1)
    return {
        "calcolo": "Quantum Error Mitigation (QEM) - Scanner Raptor GS1-128",
        "standard_crittografico": "NIST FIPS 202 (Non-Lattice) - Mitigazione
Rumore Ottico",
        "grado_qualita_stampa_iso": grado_qualita_stampa_iso,
        "contrasto_ottico_pct": contrasto,
        "affidabilita_ricostruzione_barcode_pct": affidabilita,
        "stato_qubit_mitigato": stato,
        "esito_lettura": "CONFORME ISO/IEC 15416" if affidabilita >= 80.0 else
"SCANSIONE ANOMALA"
```

```
    }
```

Applica queste correzioni ai file dal 7 al 21. NON toccare i file dall'1 al 6 (sono già corretti).